



OBD app Implementation & Integration to Android image

Submitted to: Dr Abdelrhman Abotaleb & Dr Ahmed Hussein

Team: Android

Done by:

Mohamed Ahmed Abdelazim Mohamed

&

Mohamed Ismail Sebea

Overview

- The app lets users simulate car diagnostics with a tap, showing diagnostic codes in a chat-style UI, Users can view, clear, and export logs.
- We built application using: Kivy for the graphical user interface (GUI).

Project Structure

OBD_App

main.py #Main file

diagnostics_log.json #json file where logs are saved

Project Flow

1. Launch app □ Kivy initializes the UI components □ log file path is determined and displayed.
2. User taps “Examine my car” □ triggers the diagnostic process.
3. App randomly selects 0 – 3 diagnostic codes (Errors) from a predefined list.
4. For each selected code:
 - a) Retrieves the human-readable message for the code.
 - b) Creates a new bubble with the code, message, and timestamp.
 - c) Displays the bubble in the scrollable chat area.
 - d) Prepends the code, message, and timestamp to the diagnostics_log.json file.
5. If no codes are selected:
 - a) Shows a message: “No diagnostic issues found.”
6. User taps “Export” □ app copies the log file to the device’s Download folder.
7. User taps “Clear” □ all bubbles are removed and only the log file path bubble is shown.
8. User taps “X” □ app exits.

Codes Explanation

■ Features:

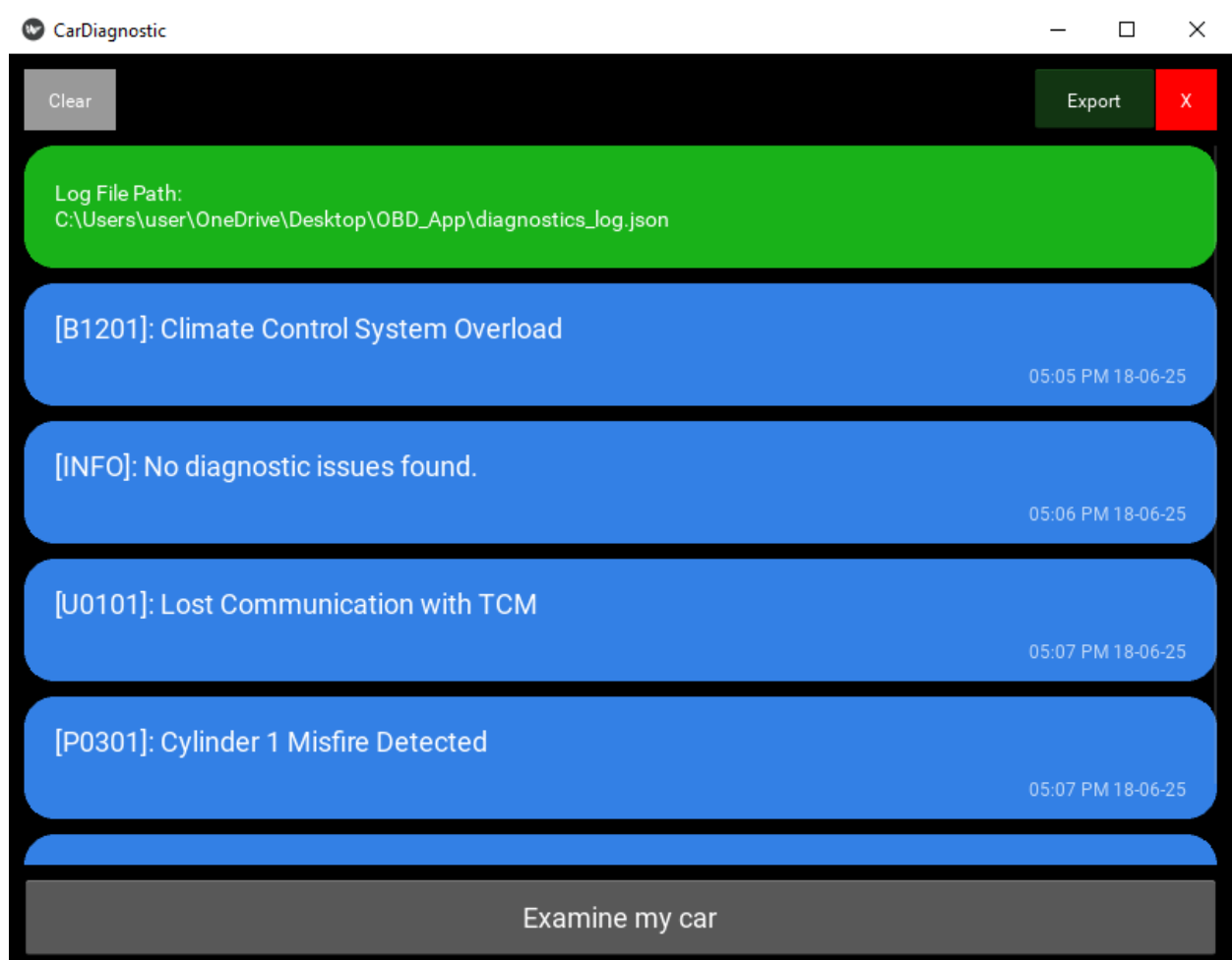
- Kivy GUI with chat-style bubbles for diagnostic messages.
- Simulates car diagnostic checks with randomized fault codes.
- Export diagnostic log to device storage (Android support).
- Clear diagnostic messages during session.
- Shows current log file path.

■ UI Layout:

- Top Bar:
 - “Clear” button: clears all displayed messages.
 - “Export” button: saves the log file to Downloads (on Android).
 - “Exit” button: closes the app.
- Center:
 - Scrollable chat area: displays diagnostic results as colorful message bubbles.
- Bottom Bar:
 - “Examine my car” button: initiates diagnostic scan and displays results.

■ Functions:

- `examine_car()` : Triggers code selection, displays bubbles, and logs messages.
- `clear_messages()` : Removes all displayed messages and resets the UI.
- `export_log_file()` : Copies `diagnostics_log.json` to the Download folder.
- `show_popup(title, message)` : Displays status or error messages in a popup window.



Converting the App to .apk file

1. Install Buildozer & some dependencies.[4]

```
sudo apt update  
  
sudo apt install -y git zip unzip openjdk-17-jdk python3-pip autoconf libtool pkg-config  
zlib1g-dev libncurses5-dev libncursesw5-dev libtinfo5 cmake libffi-dev libssl-dev  
  
pip install --upgrade pip  
  
pip install buildozer
```

2. Open project folder and initialize buildozer tool

```
cd OBD_App  
  
buildozer init
```

3. Edit buildozer.spec. [3] [2] [5]

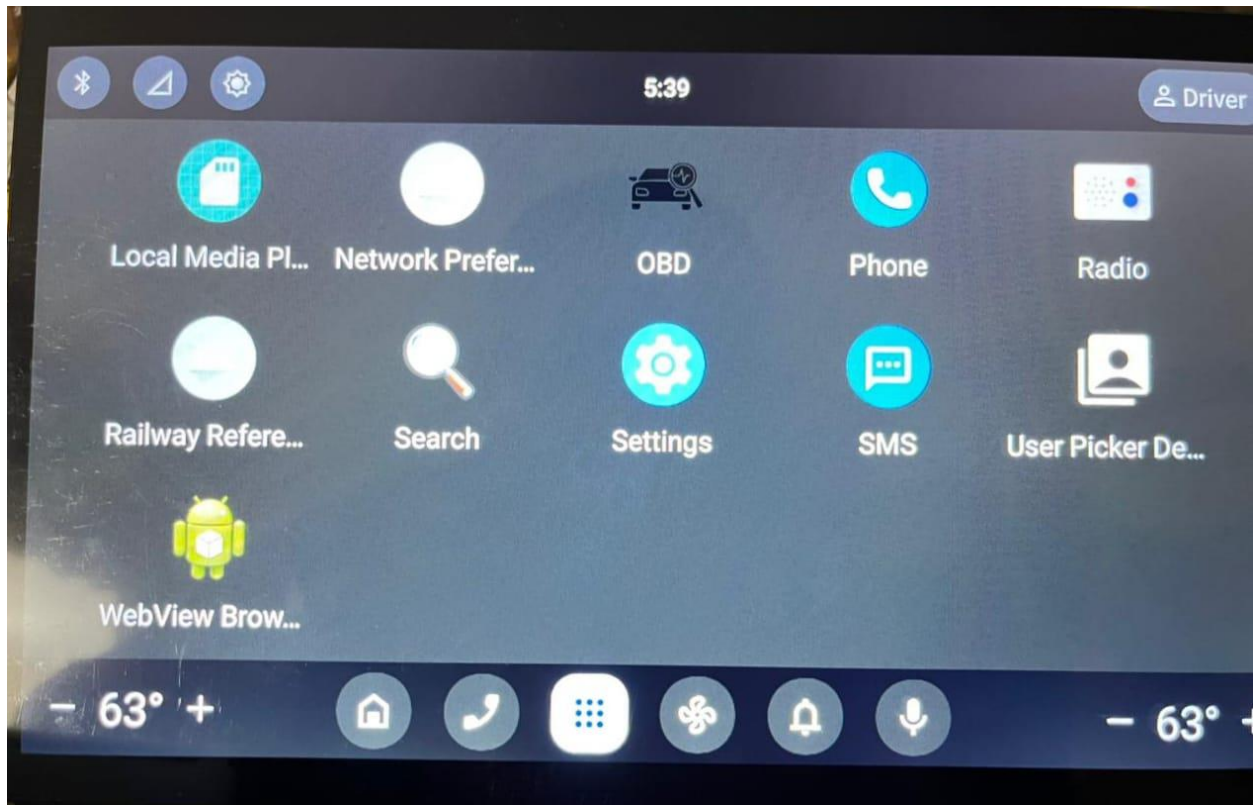
```
[app]
title = OBD
package.name = obd
package.domain = org.Abdelazim
source.dir = .
source.include_exts = py,png,jpg,kv,atlas
requirements = python3,kivy,android
orientation = landscape
fullscreen = 1
version = 1
icon.filename = diagnostics.png
android.archs = arm64-v8a
android.permissions = WRITE_EXTERNAL_STORAGE,READ_EXTERNAL_STORAGE
```

4. Build APK

```
buildozer -v android debug
```

5. Flash APK on raspi using adb command

```
adb install bin/<the bin name>
```



References

- [1] <https://docs.python.org/3/using/android.html>
- [2] <https://www.youtube.com/playlist?list=PLSiLeKadTQ7l7DJIUbCj749eAiwIHE4hH>
- [3] <https://forums.raspberrypi.com/viewtopic.php?t=362598>
- [4] <https://buildozer.readthedocs.io/en/latest/installation.html>
- [5] <https://github.com/kivy/python-for-android/blob/develop/doc/source/buildoptions.rst>